DressMe BO6 — Smart Wardrobe Digitization

Garment classification and automatic description: project report

8 October 2026 · results from the executed Kaggle notebook (run started 2026-10-08 01:13 UTC, Tesla T4)

Status: exported ML prototype, production validation pending.
From one garment photo, the prototype suggests a category among 11 classes and writes a short structured description. On catalogue validation images the selected classifier reaches **93.2% accuracy and 0.888 macro-F1** (always-predict-t-shirt baseline: 45.9% and 0.057), with the right class among its top two suggestions 99.2% of the time. The model is exported as a standalone bundle that passed 22 of 22 software checks in a fresh process. It has not been evaluated on real phone photos, and the reserved test set is still closed.

1. Problem and scope

Entering every garment into a digital wardrobe by hand is slow. BO6 lets the user upload a photo, receive a suggested category and description, correct them, and save. The two mandatory features are **garment category classification** and **automatic garment description**.

Version 1 assumes one mostly visible garment per image, and the user reviews every prediction. Multi-garment detection, segmentation, label OCR, the review-and-save interface, wardrobe storage and the BO7 integration are outside this work.

2. What I did, step by step

Step	What it does	Why
1. Audit the data	Parses the metadata, repairs malformed rows, decodes every image.	Training on broken rows or corrupt images gives numbers that mean nothing.
2. Control leakage	Finds identical images and keeps each group in one split.	The same picture in training and validation inflates the score.
3. Define the classes	Maps catalogue labels to 11 DressMe classes; lists what is unsupported.	The catalogue has 143 labels; the app needs a small, coherent set.
4. Split 70/15/15	Train / validation / reserved test, per class, by image group.	Validation guides choices; the test set is opened once at the end.
5. Train three models	Transfer learning from ImageNet: head first, then the last blocks.	Reuses learned visual features; trains in minutes.
6. Compare and select	Macro-F1 on validation, with a seed replicate and a small tuning search.	Macro-F1 weights rare classes equally; accuracy alone hides them.
7. Analyse errors	Per-class metrics, confusion matrix, error images.	Shows whether the limit is the model, the labels or the taxonomy.
8. Describe	Predicts sleeve length and pattern; suggests colour.	Second mandatory feature.
9. Export and verify	Standalone bundle, re-run in a fresh process.	The backend must use it without the notebook.

3. Data audit

The source is the Fashion Product Images dataset (Kaggle). The package contains two copies of the data; one is selected so nothing is counted twice.

Item	Measured result
Metadata records	44,446: 44,424 well-formed and 22 malformed (a comma inside the product name). All 22 were repaired and confirmed against the per-product JSON.
Images	44,441 files, all fully decoded with no failure. 5 metadata rows have no image; no orphan images.
Exact duplicates	652 groups of byte-identical images (779 redundant copies). 7 groups carry conflicting labels; their 5 rows in the mapped classes are excluded.
Classes	11: dress, jacket, jeans, shirt, shorts, skirt, sweater, sweatshirt, t-shirt, top, trousers (catalogue Track Pants are merged into trousers).
Modelling pool	15,441 images: 10,795 train / 2,310 validation / 2,336 reserved test.
Imbalance	4,931 t-shirts against 88 skirts in training (ratio 56). Moderate class weights are used.

The taxonomy is provisional: it was assigned from catalogue label strings, not from a visual review. Large catalogue labels such as Kurtas are deliberately left unsupported, not forced into a class.

4. Split and leakage: a problem found and handled

My first automatic near-duplicate detector used a perceptual hash. On this catalogue it matched *different garments worn by the same model in the same pose*, and it flagged so many pairs that the notebook's safety gate blocked training. I inspected the flagged pairs, confirmed they were mostly not duplicates, and changed the protocol:

- Images that are byte-identical (SHA-256) never cross splits. This is guaranteed and checked.
- Pixel-similar pairs are **measured, not removed**, and their effect on the validation score is reported.

Validation subset	Images	Accuracy	Macro-F1
All validation images	2,310	0.9316	0.8882
Without a near-identical training image	2,273	0.9309	0.8879
Without any pixel-similar training image	1,672	0.9222	0.8788

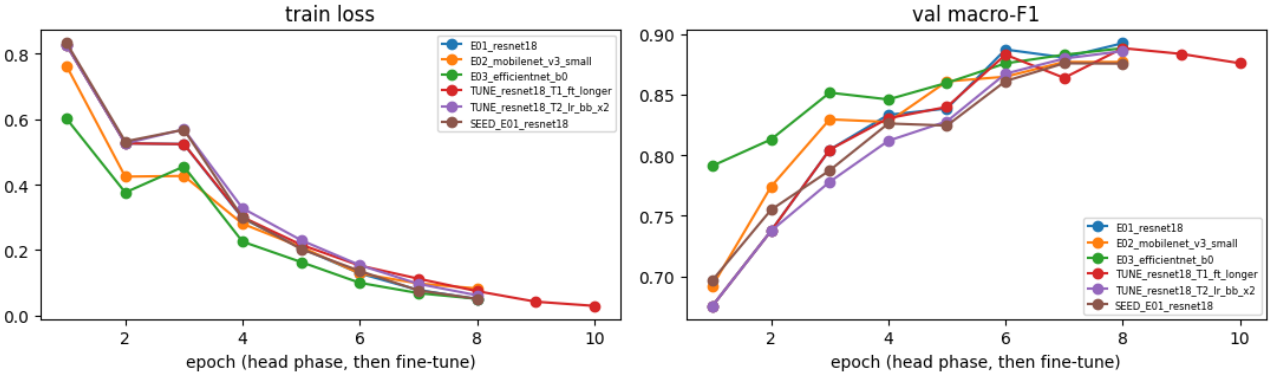
Removing every validation image that resembles a training image lowers accuracy and macro-F1 by about 0.9 points. That is smaller than the difference between two training seeds (1.7 points, section 5), so the split does not appear to inflate the result materially. The similarity search is incomplete and similar pixels are not product identity, so this limits the risk; it does not remove it.

5. Models and results

Three ImageNet-pretrained architectures were trained with the same data, preprocessing and schedule: two epochs for the new classification head with the backbone frozen, then up to six epochs fine-tuning the last blocks. Images are padded to a square and resized to 224 pixels so the whole garment is kept.

Model	Macro-F1	Accuracy	Top-2 acc.	Weakest class F1	Size	CPU time*
Majority baseline	0.057	45.9%	–	–	–	–
ResNet-18	0.892	92.9%	98.9%	0.782	44.8 MB	30.6 ms
EfficientNet-B0 (selected)	**0.888**	**93.2%**	**99.2%**	0.771	16.4 MB	28.2 ms
MobileNetV3-Small	0.877	91.4%	98.6%	0.751	6.3 MB	10.9 ms

*Model forward pass only, one image, on the Kaggle CPU. Not a server benchmark.



Training loss and validation macro-F1 per epoch. Training loss keeps falling while validation flattens after about epoch 6.

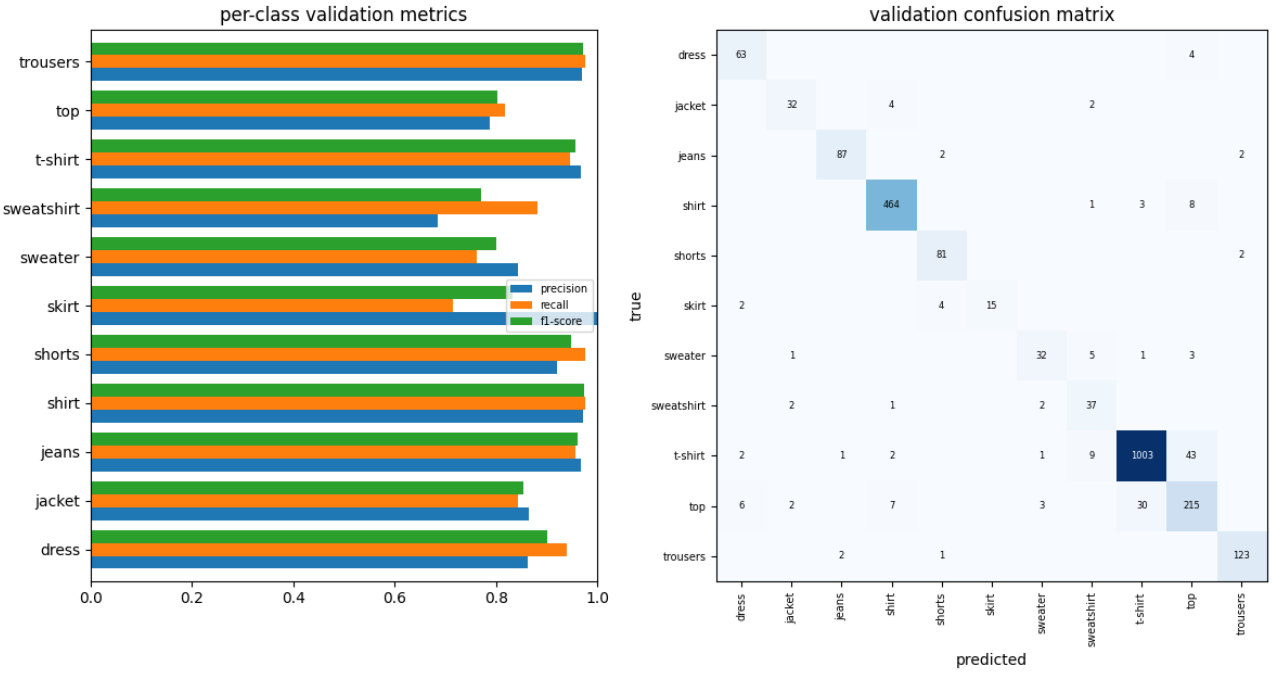
Selection. The rule was fixed in advance: take the best validation macro-F1, then prefer the fastest model within 0.01 of it. EfficientNet-B0 qualifies and is almost three times smaller than ResNet-18.

How sure is the ranking? Retraining ResNet-18 with a different seed gave 0.876 against 0.892, a gap of 0.017. The three architectures are closer to each other than that, so on this evidence they are effectively tied. MobileNetV3-Small, three times faster on CPU, is a reasonable alternative if serving speed matters.

Tuning. Two one-factor trials (longer fine-tuning, a doubled backbone learning rate) gave no gain, so the search stopped early. The limit is not the optimiser or the model size.

Reproducibility. The notebook was executed twice from a fresh kernel; the training logs and validation metrics of the two runs are identical to four decimals.

6. Error analysis



Per-class precision, recall and F1 (left) and the validation confusion matrix (right) for the selected model.

- **T-shirt versus top is the main confusion:** 73 of the 158 validation errors (43 t-shirts predicted as top, 30 the other way). The catalogue seems to separate these two largely by the wearer's gender, not by shape, so this is a taxonomy question more than a model weakness.
- **Weakest classes:** sweatshirt (F1 0.77; precision 0.69), sweater (0.80), top (0.80), and skirt (0.83 with only 21 validation images, so the estimate is noisy).

- **Some errors are label noise.** In the high-score error images below, several catalogue labels look wrong: denim shorts labelled jeans, shorts labelled trousers, a buttoned shirt labelled Tops.
- **A high score is not proof.** 88 of the 158 errors have a score above 0.8. Scores are raw softmax values, not calibrated probabilities.



Validation errors with the highest scores, shown as catalogue label → prediction.

Review threshold. On validation, predictions scoring at least 0.7 cover 95.4% of images with 5.0% error. The threshold was chosen on the same images it is reported on, so it is slightly optimistic, and every prediction remains flagged for user review.

7. Automatic description

The description is a **deterministic template over structured fields**, not text produced by a generative language model. A first version guessed the colour from the centre of the image; it agreed with the catalogue only 39% of the time, so I removed it. The current version adds small classifiers (“attribute heads”) on top of the selected model's features, trained on labels already present in the catalogue.

Attribute	Garment types where it applies	Labelled validation images	Agreement with catalogue	Policy in the description
Sleeve length	jacket, shirt, sweatshirt, t-shirt, top	1,639	92.7% (baseline 67.6%)	Stated. Error 7.3% on these images.
Pattern	all except jeans and sweater	1,835	82.7% (baseline 42.7%)	Stated only when the score is at least 0.97: 72.5% of these images answered, 9.3% error.
Colour	all 11 classes	2,291	69.1% (baseline 17.4%)	Never stated. Two suggestions are offered for the user to confirm.

These figures are agreement with catalogue metadata on the images that carry a usable label; they are not human-checked truth. Missing labels were never treated as a value, and unsupported values (188 three-quarter sleeves, rare patterns) were excluded. The thresholds were explored on the same validation images, so the errors above are optimistic. “Sleeveless” is the weakest sleeve value (precision 0.67).

End to end, on all 2,310 validation images with predicted categories:

- 1,310 descriptions state two attributes and 682 state one, for example “A printed short-sleeved t-shirt.” or “A checked long-sleeved shirt.”
- 318 state the category only, for example “A pair of trousers.”

- 106 withhold the category because the score is below 0.7 and read “A garment.”
- No description states a colour without user confirmation.

These are counts of what the template writes. Whether the stated attributes are true needs a human review, which has not been done.

8. The exported bundle

`selected_model_bundle.zip` holds 25 files in 15.3 MB (including the 2 files of the optional quality assistant, section 8b): the EfficientNet-B0 weights, the attribute heads, the class list, the preprocessing settings, the review policy, a README, a model card and checksums. The backend needs one call:

```
from attributes import GarmentDescriber
result = GarmentDescriber().predict("photo.jpg")
result = GarmentDescriber().predict("photo.jpg", confirmed={"color": "blue"})   # after a user correction
```

The result contains the category, its score, the top two suggestions, a review flag, the description, the structured fields, where each field came from (classifier, attribute model or user), and colour suggestions. `inference.GarmentClassifier` remains as a category-only fallback.

Verification. The bundle's own self-test ran in a fresh Python process and passed 22 of 22 checks on 3 synthetic and 44 validation images: identical classifier outputs and attribute probabilities, identical text and fields, no attribute returned for garment types it does not apply to, a withheld category at low score, user corrections taking priority, plural wording, and a clean error for corrupt or truncated images. This shows the exported code behaves as specified. It says nothing about accuracy.



Six validation images through the exported classifier (screenshot from the earlier category-only version of the bundle).

The same six photos through the final bundle. I unpacked the downloaded `selected_model_bundle.zip` on a different machine (CPU only, a newer PyTorch version than Kaggle), confirmed all 25 checksums, ran its self-test (17 of 17 portable checks passed) and called `GarmentDescriber().predict()` on crops of the six photos above:

Catalogue label	Prediction (score)	2nd suggestion	Description	Colour suggestions	Quality warnings
dress	dress (1.00)	top	A dress.	black, pink	low resolution
jacket	**shirt (0.82)**	top	A long-sleeved shirt.	blue, purple	low resolution

Catalogue label	Prediction (score)	2nd suggestion	Description	Colour suggestions	Quality warnings
jeans	jeans (1.00)	trousers	A pair of jeans.	blue, black	low resolution
shirt	shirt (1.00)	jacket	A checked long-sleeved shirt.	green, grey	low resolution
shorts	shorts (1.00)	trousers	A pair of shorts.	blue, navy	none
skirt	skirt (1.00)	dress	A skirt.	yellow, red	low resolution

The crops come from a screenshot, so most are under 224 pixels and the quality assistant correctly flags them as low resolution. The coat is still predicted as a shirt, now with sleeves added to the sentence: the description inherits the classifier's mistakes, which is why the user reviews every result.

8b. Optional enhancement: image quality assistant

Before a photo is classified, `quality_checks.check_image(photo)` reports whether it is small, blurry, very dark, washed out or low in contrast, each with a suggested action (for example “Hold the phone steady, tap to focus, and retake.”). It uses image-processing rules, not a trained model, and its warnings are advisory: they never block a prediction.

The blur threshold was calibrated on 300 validation photos and blurred copies of them; the other thresholds are fixed starting values. I then tested the rules in a fresh process on 60 clear validation photos and four synthetic degradations of each:

Image set	Expected warning	Share with the expected warning	Share with any warning
Clear catalogue photos	none	–	0%
Blurred	blurry	100%	100%
Darkened	too dark	100%	100%
Washed out	too bright	100%	100%
Downscaled	low resolution	100%	100%



One validation photo and its four synthetic degradations, with the warnings returned.

These degradations are strong on purpose, so a perfect score here shows the rules work as written, not that they catch real phone problems. The blur threshold (sharpness below 13.8) sits far below the clear photos (5th percentile 122.6), so mild blur will not be flagged yet. All thresholds stay provisional until they are checked on DressMe phone photos. The checks also cannot tell whether the garment is fully visible.

9. What I can and cannot claim

Supported by evidence	Not supported yet
Good provisional performance on catalogue validation images.	Accuracy on the reserved test set (not opened).
A reproducible pipeline: two full runs, identical metrics.	Accuracy on real DressMe phone photos.
Sleeve and pattern predictions that agree with catalogue labels at the stated rates.	Factual correctness of descriptions as judged by a person.
A standalone bundle whose behaviour is verified in a fresh process.	Deployment reliability, or any saving of user time.

One gap deserves emphasis: most catalogue photos show a model wearing the garment on a plain background, while DressMe expects garments laid flat or hanging, photographed by phone. Catalogue scores say nothing about that setting.

10. Next steps

- Collect and label DressMe phone photos (10–20 per class to start), keeping a separate set untouched for the final evaluation.
- Review the t-shirt / top boundary and decide whether to merge the two classes.
- Have a person review a sample of descriptions to measure factual correctness.
- Freeze the model and thresholds, then open the reserved test set once.
- Install the bundle in the backend, run its self-test there, and build the upload → review → save flow.
- Confirm the dataset licence before any redistribution.

Glossary

Term	Meaning here
Transfer learning	Starting from a network already trained on ImageNet and adapting it to garments, instead of training from zero.
Macro-F1	The average of each class's F1 score, so a rare class such as skirt counts as much as t-shirt.
Data leakage	The same or nearly the same image appearing in both training and evaluation data, which inflates the score.
Class weighting	Giving rare classes more weight in the loss so the model does not ignore them.
Softmax score	The model's raw confidence value between 0 and 1. It is not a calibrated probability of being right.
Weak labels	Labels taken from catalogue metadata without human verification.